

## CHAPTER

## 2

## Regular Expressions, Tokenization, Edit Distance

```

User: I am unhappy.
ELIZA: DO YOU THINK COMING HERE WILL HELP YOU NOT TO BE UNHAPPY
User: I need some help, that much seems certain.
ELIZA: WHAT WOULD IT MEAN TO YOU IF YOU GOT SOME HELP
User: Perhaps I could learn to get along with my mother.
ELIZA: TELL ME MORE ABOUT YOUR FAMILY
User: My mother takes care of me.
ELIZA: WHO ELSE IN YOUR FAMILY TAKES CARE OF YOU
User: My father.
ELIZA: YOUR FATHER
User: You are like my father in some ways.

```

Weizenbaum (1966)

## ELIZA

The dialogue above is from **ELIZA**, an early natural language processing system that could carry on a limited conversation with a user by imitating the responses of a Rogerian psychotherapist (Weizenbaum, 1966). ELIZA is a surprisingly simple program that uses pattern matching to recognize phrases like “I need X” and translate them into suitable outputs like “What would it mean to you if you got X?”. This simple technique succeeds in this domain because ELIZA doesn’t actually need to *know* anything to mimic a Rogerian psychotherapist. As Weizenbaum notes, this is one of the few dialogue genres where listeners can act as if they know nothing of the world. ELIZA’s mimicry of human conversation was remarkably successful: many people who interacted with ELIZA came to believe that it really *understood* them and their problems, many continued to believe in ELIZA’s abilities even after the program’s operation was explained to them (Weizenbaum, 1976), and even today such **chatbots** are a fun diversion.

## chatbots

Of course modern conversational agents are much more than a diversion; they can answer questions, book flights, or find restaurants, functions for which they rely on a much more sophisticated understanding of the user’s intent, as we will see in Chapter 15. Nonetheless, the simple pattern-based methods that powered ELIZA and other chatbots play a crucial role in natural language processing.

We’ll begin with the most important tool for describing text patterns: the **regular expression**. Regular expressions can be used to specify strings we might want to extract from a document, from transforming “I need X” in ELIZA above, to defining strings like \$199 or \$24.99 for extracting tables of prices from a document.

text  
normalization

We’ll then turn to a set of tasks collectively called **text normalization**, in which regular expressions play an important part. Normalizing text means converting it to a more convenient, standard form. For example, most of what we are going to do with language relies on first separating out or **tokenizing** words or word parts from running text, the task of **tokenization**. English words are often separated from each other by whitespace, but whitespace is not always sufficient. *New York* and *rock ‘n’ roll* are sometimes treated as large words despite the fact that they contain spaces, while sometimes we’ll need to separate *I’m* into the two words *I* and *am*. For processing tweets or texts we’ll need to tokenize **emoticons** like :) or **hashtags**

## tokenization

like `#nlp`. Some languages, like Japanese, don't have spaces between words, so word tokenization becomes more difficult. And as we'll see, for large language models we'll use tokens that range greatly in size, from letters to subwords (parts of words) to words and even sometimes short phrases.

lemmatization

Another part of text normalization is **lemmatization**, the task of determining that two words have the same root, despite their surface differences. For example, the words *sang*, *sung*, and *sings* are forms of the verb *sing*. The word *sing* is the common *lemma* of these words, and a **lemmatizer** maps from all of these to *sing*. Lemmatization is essential for processing morphologically complex languages like Arabic. **Stemming** refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word. Text normalization also includes **sentence segmentation**: breaking up a text into individual sentences, using cues like periods or exclamation points.

stemming

sentence  
segmentation

Finally, we'll need to compare words and other strings. We'll introduce a metric called **edit distance** that measures how similar two strings are based on the number of edits (insertions, deletions, substitutions) it takes to change one string into the other. Edit distance is an algorithm with applications throughout language processing, from spelling correction to speech recognition to coreference resolution.

## 2.1 Regular Expressions

regular  
expression

One of the most useful tools for text processing in computer science has been the **regular expression** (often shortened to **regex**), a language for specifying text search strings. This practical language is used in every computer language, in text processing tools like the Unix tools `grep`, and in editors like `vim` or `Emacs`. Formally, a regular expression is an algebraic notation for characterizing a set of strings. Regular expressions are particularly useful for searching in texts, when we have a **pattern** to search for and a **corpus** of texts to search through. A regular expression search function will search through the corpus, returning all texts that match the pattern. The corpus can be a single document or a collection. For example, the Unix command-line tool `grep` takes a regular expression and returns every line of the input document that matches the expression.

corpus

A search can be designed to return every match on a line, if there are more than one, or just the first match. In the following examples we generally underline the exact string that matches the regular expression and show only the first match. We'll show regular expressions delimited by slashes but note that slashes are *not* part of the regular expressions.

Regular expressions come in many variants. We'll be describing **extended regular expressions**; different regular expression parsers may only recognize subsets of these, or treat some expressions slightly differently. Using an online regular expression tester is a handy way to test out your expressions and explore these variations.

### 2.1.1 Basic Regular Expression Patterns

concatenation

The simplest kind of regular expression is a sequence of simple characters; putting characters in sequence is called **concatenation**. To search for *woodchuck*, we type `/woodchuck/`. The expression `/Buttercup/` matches any string containing the substring *Buttercup*; `grep` with that expression would return the line *I'm called little Buttercup*. The search string can consist of a single character (like `/!/`) or a

sequence of characters (like `/urgl/`) (see Fig. 2.1).

| Regex                     | Example Patterns Matched                          |
|---------------------------|---------------------------------------------------|
| <code>/woodchucks/</code> | “interesting links to woodchucks and lemurs”      |
| <code>/a/</code>          | “Mary Ann stopped by Mona’s”                      |
| <code>/!/</code>          | “You’ve left the burglar behind again!” said Nori |

**Figure 2.1** Some simple regex searches.

Regular expressions are **case sensitive**; lower case `/s/` is distinct from upper case `/S/` (`/s/` matches a lower case *s* but not an upper case *S*). This means that the pattern `/woodchucks/` will not match the string *Woodchucks*. We can solve this problem with the use of the square braces `[` and `]`. The string of characters inside the braces specifies a **disjunction** of characters to match. For example, Fig. 2.2 shows that the pattern `/[wW]/` matches patterns containing either *w* or *W*.

| Regex                       | Match                  | Example Patterns                 |
|-----------------------------|------------------------|----------------------------------|
| <code>/[wW]oodchuck/</code> | Woodchuck or woodchuck | “Woodchuck”                      |
| <code>/[abc]/</code>        | ‘a’, ‘b’, or ‘c’       | “In uomini, in soldat <u>i</u> ” |
| <code>/[1234567890]/</code> | any digit              | “plenty of <u>7</u> to 5”        |

**Figure 2.2** The use of the brackets `[]` to specify a disjunction of characters.

The regular expression `/[1234567890]/` specifies any single digit. While such classes of characters as digits or letters are important building blocks in expressions, they can get awkward (e.g., it’s inconvenient to specify

$$/[ABCDEFGHIJKLMNOPQRSTUVWXYZ]/ \quad (2.1)$$

to mean “any capital letter”). In cases where there is a well-defined sequence associated with a set of characters, the brackets can be used with the dash `-` to specify any one character in a **range**. The pattern `/[2-5]/` specifies any one of the characters 2, 3, 4, or 5. The pattern `/[b-g]/` specifies one of the characters *b*, *c*, *d*, *e*, *f*, or *g*. Some other examples are shown in Fig. 2.3.

range

| Regex                | Match                | Example Patterns Matched                       |
|----------------------|----------------------|------------------------------------------------|
| <code>/[A-Z]/</code> | an upper case letter | “we should call it ‘Drenched Blossoms’ ”       |
| <code>/[a-z]/</code> | a lower case letter  | “ <u>m</u> y beans were impatient to be hoed!” |
| <code>/[0-9]/</code> | a single digit       | “Chapter <u>1</u> : Down the Rabbit Hole”      |

**Figure 2.3** The use of the brackets `[]` plus the dash `-` to specify a range.

The square braces can also be used to specify what a single character *cannot* be, by use of the caret `^`. If the caret `^` is the first symbol after the open square brace `[`, the resulting pattern is negated. For example, the pattern `/[^a]/` matches any single character (including special characters) except *a*. This is only true when the caret is the first symbol after the open square brace. If it occurs anywhere else, it usually stands for a caret; Fig. 2.4 shows some examples.

How can we talk about optional elements, like an optional *s* in *woodchuck* and *woodchucks*? We can’t use the square brackets, because while they allow us to say “s or S”, they don’t allow us to say “s or nothing”. For this we use the question mark `/?/`, which means “the preceding character or nothing”, as shown in Fig. 2.5.

We can think of the question mark as meaning “zero or one instances of the previous character”. That is, it’s a way of specifying how many of something that

| Regex                 | Match (single characters)      | Example Patterns Matched              |
|-----------------------|--------------------------------|---------------------------------------|
| /[ <sup>^</sup> A-Z]/ | not an upper case letter       | “Oyfn pripetchik”                     |
| /[ <sup>^</sup> Ss]/  | neither ‘S’ nor ‘s’            | “I have no exquisite reason for’t”    |
| /[ <sup>^</sup> .]/   | not a period                   | “ <u>o</u> ur resident Djinn”         |
| /[e <sup>^</sup> ]/   | either ‘e’ or ‘ <sup>^</sup> ’ | “look up <u>^</u> now”                |
| /a <sup>^</sup> b/    | the pattern ‘a <sup>^</sup> b’ | “look up <u>a</u> <sup>^</sup> b now” |

**Figure 2.4** The caret <sup>^</sup> for negation or just to mean <sup>^</sup>. See below re: the backslash for escaping the period.

| Regex         | Match                   | Example Patterns Matched |
|---------------|-------------------------|--------------------------|
| /woodchucks?/ | woodchuck or woodchucks | “ <u>woodchuck</u> ”     |
| /colou?r/     | color or colour         | “ <u>color</u> ”         |

**Figure 2.5** The question mark ? marks optionality of the previous expression.

we want, something that is very important in regular expressions. For example, consider the language of certain sheep, which consists of strings that look like the following:

baa!  
baaa!  
baaaa!  
...

This language consists of strings with a *b*, followed by at least two *a*’s, followed by an exclamation point. The set of operators that allows us to say things like “some number of *as*” are based on the asterisk or \*, commonly called the **Kleene \*** (generally pronounced “cleany star”). The Kleene star means “zero or more occurrences of the immediately previous character or regular expression”. So */a\*/* means “any string of zero or more *as*”. This will match *a* or *aaaaaa*, but it will also match the empty string at the start of *Off Minor* since the string *Off Minor* starts with zero *a*’s. So the regular expression for matching one or more *a* is */aa\*/*, meaning one *a* followed by zero or more *as*. More complex patterns can also be repeated. So */[ab]\** means “zero or more *a*’s or *b*’s” (not “zero or more right square braces”). This will match strings like *aaaa* or *ababab* or *bbbb*, as well as the empty string.

**Kleene \***

For specifying multiple digits (useful for finding prices) we can extend */[0-9]/*, the regular expression for a single digit. An integer (a string of digits) is thus */[0-9][0-9]\**/. (Why isn’t it just */[0-9]\**/?)

**Kleene +**

Sometimes it’s annoying to have to write the regular expression for digits twice, so there is a shorter way to specify “at least one” of some character. This is the **Kleene +**, which means “one or more occurrences of the immediately preceding character or regular expression”. Thus, the expression */[0-9]+* is the normal way to specify “a sequence of digits”. There are thus two ways to specify the sheep language: */baaa\*/*! or */baa+!*!.

One very important special character is the period (*/./*), a **wildcard** expression that matches any single character (*except* a carriage return), as shown in Fig. 2.6.

| Regex   | Match                                         | Example Matches                            |
|---------|-----------------------------------------------|--------------------------------------------|
| /beg.n/ | any character between <i>beg</i> and <i>n</i> | <u>begin</u> , <u>beg’n</u> , <u>begun</u> |

**Figure 2.6** The use of the period . to specify any character.

The wildcard is often used together with the Kleene star to mean “any string of characters”. For example, suppose we want to find any line in which a particular

word, for example, *aardvark*, appears twice. We can specify this with the regular expression `/aardvark.*aardvark/`.

**anchors** **anchors** are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret `^` and the dollar sign `$`. The caret `^` matches the start of a line. The pattern `/^The/` matches the word *The* only at the start of a line. Thus, the caret `^` has three uses: to match the start of a line, to indicate a negation inside of square brackets, and just to mean a caret. (What are the contexts that allow `grep` or Python to know which function a given caret is supposed to have?) The dollar sign `$` matches the end of a line. So the pattern `␣$` is a useful pattern for matching a space at the end of a line, and `/^The dog\.$/` matches a line that contains only the phrase *The dog*. (We have to use the backslash here since we want the `.` to mean “period” and not the wildcard.)

| Regex           | Match             |
|-----------------|-------------------|
| <code>^</code>  | start of line     |
| <code>\$</code> | end of line       |
| <code>\b</code> | word boundary     |
| <code>\B</code> | non-word boundary |

**Figure 2.7** Anchors in regular expressions.

There are also two other anchors: `\b` matches a word boundary, and `\B` matches a non word-boundary. Thus, `/\bthe\b/` matches the word *the* but not the word *other*. A “word” for the purposes of a regular expression is defined based on the definition of words in programming languages as a sequence of digits, underscores, or letters. Thus `/\b99\b/` will match the string *99* in *There are 99 bottles of beer on the wall* (because *99* follows a space) but not *99* in *There are 299 bottles of beer on the wall* (since *99* follows a number). But it will match *99* in *\$99* (since *99* follows a dollar sign (`$`), which is not a digit, underscore, or letter).

2.1.2 Disjunction, Grouping, and Precedence

**disjunction** Suppose we need to search for texts about pets; perhaps we are particularly interested in cats and dogs. In such a case, we might want to search for either the string *cat* or the string *dog*. Since we can’t use the square brackets to search for “cat or dog” (why can’t we say `/[catdog]/?`), we need a new operator, the **disjunction** operator, also called the **pipe** symbol `|`. The pattern `/cat|dog/` matches either the string *cat* or the string *dog*.

**precedence** Sometimes we need to use this disjunction operator in the midst of a larger sequence. For example, suppose I want to search for information about pet fish for my cousin David. How can I specify both *guppy* and *guppies*? We cannot simply say `/guppy|ies/`, because that would match only the strings *guppy* and *ies*. This is because sequences like *guppy* take **precedence** over the disjunction operator `|`. To make the disjunction operator apply only to a specific pattern, we need to use the parenthesis operators `(` and `)`. Enclosing a pattern in parentheses makes it act like a single character for the purposes of neighboring operators like the pipe `|` and the Kleene\*. So the pattern `/gupp(y|ies)/` would specify that we meant the disjunction only to apply to the suffixes *y* and *ies*.

The parenthesis operator `(` is also useful when we are using counters like the Kleene\*. Unlike the `|` operator, the Kleene\* operator applies by default only to a single character, not to a whole sequence. Suppose we want to match repeated instances of a string. Perhaps we have a line that has column labels of the form

*Column 1 Column 2 Column 3.* The expression `/Column_+[0-9]+_*/` will not match any number of columns; instead, it will match a single column followed by any number of spaces! The star here applies only to the space `_` that precedes it, not to the whole sequence. With the parentheses, we could write the expression `/(Column_+[0-9]+_)*` to match the word *Column*, followed by a number and optional spaces, the whole pattern repeated zero or more times.

operator  
precedence

This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the **operator precedence hierarchy** for regular expressions. The following table gives the order of RE operator precedence, from highest precedence to lowest precedence.

|                       |               |
|-----------------------|---------------|
| Parenthesis           | ()            |
| Counters              | * + ? {}      |
| Sequences and anchors | the ^my end\$ |
| Disjunction           |               |

Thus, because counters have a higher precedence than sequences, `/the*/` matches *theeee* but not *thethe*. Because sequences have a higher precedence than disjunction, `/the|any/` matches *the* or *any* but not *thany* or *theny*.

Patterns can be ambiguous in another way. Consider the expression `/[a-z]*/` when matching against the text *once upon a time*. Since `/[a-z]*/` matches zero or more letters, this expression could match nothing, or just the first letter *o*, *on*, *onc*, or *once*. In these cases regular expressions always match the *largest* string they can; we say that patterns are **greedy**, expanding to cover as much of a string as they can.

greedy  
non-greedy

There are, however, ways to enforce **non-greedy** matching, using another meaning of the `?` qualifier. The operator `*?` is a Kleene star that matches as little text as possible. The operator `+?` is a Kleene plus that matches as little text as possible.

### 2.1.3 A Simple Example

Suppose we wanted to write a RE to find cases of the English article *the*. A simple (but incorrect) pattern might be:

`/the/` (2.2)

One problem is that this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., *The*). This might lead us to the following pattern:

`/[tT]he/` (2.3)

But we will still overgeneralize, incorrectly return texts with **the** embedded in other words (e.g., *other* or *there*). So we need to specify that we want instances with a word boundary on both sides:

`/\b[tT]he\b/` (2.4)

false positives  
false negatives

The simple process we just went through was based on fixing two kinds of errors: **false positives**, strings that we incorrectly matched like *other* or *there*, and **false negatives**, strings that we incorrectly missed, like *The*. Addressing these two kinds of errors comes up again and again in language processing. Reducing the overall error rate for an application thus involves two antagonistic efforts:

- Increasing **precision** (minimizing false positives)
- Increasing **recall** (minimizing false negatives)

We'll come back to precision and recall with more precise definitions in Chapter 4.

### 2.1.4 More Operators

Figure 2.8 shows some aliases for common ranges, which can be used mainly to save typing. Besides the Kleene  $*$  and Kleene  $+$  we can also use explicit numbers as counters, by enclosing them in curly brackets. The operator  $/\{3\}/$  means “exactly 3 occurrences of the previous character or expression”. So  $/a\.{24}z/$  will match  $a$  followed by 24 dots followed by  $z$  (but not  $a$  followed by 23 or 25 dots followed by a  $z$ ).

| Regex           | Expansion                 | Match                       | First Matches |
|-----------------|---------------------------|-----------------------------|---------------|
| <code>\d</code> | <code>[0-9]</code>        | any digit                   | Party_of_5    |
| <code>\D</code> | <code>[^0-9]</code>       | any non-digit               | Blue_moon     |
| <code>\w</code> | <code>[a-zA-Z0-9_]</code> | any alphanumeric/underscore | Daiyu         |
| <code>\W</code> | <code>[^\w]</code>        | a non-alphanumeric          | !!!!          |
| <code>\s</code> | <code>[\r\t\n\f]</code>   | whitespace (space, tab)     | in_Concord    |
| <code>\S</code> | <code>[^\s]</code>        | Non-whitespace              | in_Concord    |

**Figure 2.8** Aliases for common sets of characters.

A range of numbers can also be specified. So  $/\{n,m\}/$  specifies from  $n$  to  $m$  occurrences of the previous char or expression, and  $/\{n,\}/$  means at least  $n$  occurrences of the previous expression. REs for counting are summarized in Fig. 2.9.

| Regex              | Match                                                          |
|--------------------|----------------------------------------------------------------|
| <code>*</code>     | zero or more occurrences of the previous char or expression    |
| <code>+</code>     | one or more occurrences of the previous char or expression     |
| <code>?</code>     | zero or one occurrence of the previous char or expression      |
| <code>{n}</code>   | exactly $n$ occurrences of the previous char or expression     |
| <code>{n,m}</code> | from $n$ to $m$ occurrences of the previous char or expression |
| <code>{n,}</code>  | at least $n$ occurrences of the previous char or expression    |
| <code>{,m}</code>  | up to $m$ occurrences of the previous char or expression       |

**Figure 2.9** Regular expression operators for counting.

Finally, certain special characters are referred to by special notation based on the backslash (`\`) (see Fig. 2.10). The most common of these are the **newline** character `\n` and the **tab** character `\t`. To refer to characters that are special themselves (like `.`, `*`, `[`, and `\`), precede them with a backslash, (i.e., `\.`, `\*`, `\[`, and `\\`).

| Regex           | Match                          | First Patterns Matched                                |
|-----------------|--------------------------------|-------------------------------------------------------|
| <code>\*</code> | an asterisk “ <code>*</code> ” | “ <code>K*A*P*L*A*N</code> ”                          |
| <code>\.</code> | a period “ <code>.</code> ”    | “ <code>Dr. Livingston, I presume</code> ”            |
| <code>\?</code> | a question mark                | “ <code>Why don’t they come and lend a hand?</code> ” |
| <code>\n</code> | a newline                      |                                                       |
| <code>\t</code> | a tab                          |                                                       |

**Figure 2.10** Some characters that need to be escaped (via backslash).

### 2.1.5 A More Complex Example

Let’s try out a more significant example of the power of REs. Suppose our goal is help a user buy a computer on the Web who wants “at least 6 GHz and 500 GB of disk space for less than \$1000”. To do this kind of retrieval, we first need to be

able to look for expressions like *6 GHz* or *500 GB* or *\$999.99*. Let's work out some regular expressions for this task.

First, let's complete our regular expression for prices. Here's a regular expression for a dollar sign followed by a string of digits:

```
/$[0-9]+/ (2.5)
```

Note that the \$ character has a different function here than the end-of-line function we discussed earlier. Most regular expression parsers are smart enough to realize that \$ here doesn't mean end-of-line. (As a thought experiment, think about how regex parsers might figure out the function of \$ from the context.)

Now we just need to deal with fractions of dollars. We'll add a decimal point and two digits afterwards:

```
/$[0-9]+\.[0-9][0-9]/ (2.6)
```

This pattern only allows *\$199.99* but not *\$199*. We need to make the cents optional and to make sure we're at a word boundary:

```
/(^\W)$[0-9]+(\.[0-9][0-9])?\b/ (2.7)
```

One last catch! This pattern allows prices like *\$199999.99* which would be far too expensive! We need to limit the dollars:

```
/(^\W)$[0-9]{0,3}(\.[0-9][0-9])?\b/ (2.8)
```

Further fixes (like avoiding matching a dollar sign with no price after it) are left as an exercise for the reader.

How about disk space? We'll need to allow for optional fractions again (*5.5 GB*); note the use of ? for making the final s optional, and the use of /\_\*/ to mean "zero or more spaces" since there might always be extra spaces lying around:

```
/\b[0-9]+(\.[0-9]+)? *(GB|[Gg]igabytes?)\b/ (2.9)
```

Modifying this regular expression so that it only matches more than 500 GB is left as an exercise for the reader.

### 2.1.6 Substitution, Capture Groups, and ELIZA

#### substitution

An important use of regular expressions is in **substitutions**. For example, the substitution operator `s/regexp1/pattern/` used in Python and in Unix commands like `vim` or `sed` allows a string characterized by a regular expression to be replaced by another string:

```
s/colour/color/ (2.10)
```

It is often useful to be able to refer to a particular subpart of the string matching the first pattern. For example, suppose we wanted to put angle brackets around all integers in a text, for example, changing *the 35 boxes* to *the <35> boxes*. We'd like a way to refer to the integer we've found so that we can easily add the brackets. To do this, we put parentheses ( and ) around the first pattern and use the **number** operator \1 in the second pattern to refer back. Here's how it looks:

```
s/([0-9]+)/<\1>/ (2.11)
```

The parenthesis and number operators can also specify that a certain string or expression must occur twice in the text. For example, suppose we are looking for the pattern “the Xer they were, the Xer they will be”, where we want to constrain the two X’s to be the same string. We do this by surrounding the first X with the parenthesis operator, and replacing the second X with the number operator \1, as follows:

```
/the (.*)er they were, the \1er they will be/ (2.12)
```

Here the \1 will be replaced by whatever string matched the first item in parentheses. So this will match *the bigger they were, the bigger they will be* but not *the bigger they were, the faster they will be*.

capture group

This use of parentheses to store a pattern in memory is called a **capture group**. Every time a capture group is used (i.e., parentheses surround a pattern), the resulting match is stored in a numbered **register**. If you match two different sets of parentheses, \2 means whatever matched the *second* capture group. Thus

register

```
/the (.*)er they (.*) , the \1er we \2/ (2.13)
```

will match *the faster they ran, the faster we ran* but not *the faster they ran, the faster we ate*. Similarly, the third capture group is stored in \3, the fourth is \4, and so on.

Parentheses thus have a double function in regular expressions; they are used to group terms for specifying the order in which operators should apply, and they are used to capture something in a register. Occasionally we might want to use parentheses for grouping, but don’t want to capture the resulting pattern in a register. In that case we use a **non-capturing group**, which is specified by putting the special commands ?: after the open parenthesis, in the form (?: pattern ).

non-capturing group

```
/(?:some|a few) (people|cats) like some \1/ (2.14)
```

will match *some cats like some cats* but not *some cats like some some*.

Substitutions and capture groups are very useful in implementing simple chat-bots like ELIZA (Weizenbaum, 1966). Recall that ELIZA simulates a Rogerian psychologist by carrying on conversations like the following:

```
User1:   Men are all alike.
ELIZA1: IN WHAT WAY
User2:   They're always bugging us about something or other.
ELIZA2: CAN YOU THINK OF A SPECIFIC EXAMPLE
User3:   Well, my boyfriend made me come here.
ELIZA3: YOUR BOYFRIEND MADE YOU COME HERE
User4:   He says I'm depressed much of the time.
ELIZA4: I AM SORRY TO HEAR YOU ARE DEPRESSED
```

ELIZA works by having a series or cascade of regular expression substitutions each of which matches and changes some part of the input lines. After the input is uppercased, substitutions change all instances of *MY* to *YOUR*, and *I'M* to *YOU ARE*, and so on. That way when ELIZA repeats back part of the user utterance, it will seem to be referring correctly to the user. The next set of substitutions matches and replaces other patterns in the input. Here are some examples:

```
s/. * YOU ARE (depressed|sad) . */I AM SORRY TO HEAR YOU ARE \1/
s/. * YOU ARE (depressed|sad) . */WHY DO YOU THINK YOU ARE \1/
s/. * all . */IN WHAT WAY/
s/. * always . */CAN YOU THINK OF A SPECIFIC EXAMPLE/
```

Since multiple substitutions can apply to a given input, substitutions are assigned a rank and applied in order. Creating patterns is the topic of Exercise 2.3, and we return to the details of the ELIZA architecture in Chapter 15.

### 2.1.7 Lookahead Assertions

Finally, there will be times when we need to predict the future: look ahead in the text to see if some pattern matches, but not yet advance the pointer we always keep to where we are in the text, so that we can then deal with the pattern if it occurs, but if it doesn't we can check for something else instead.

**lookahead** These **lookahead** assertions make use of the `(? syntax that we saw in the previous section for non-capture groups. The operator (?= pattern) is true if pattern occurs, but is zero-width, i.e. the match pointer doesn't advance. The operator (?! pattern) only returns true if a pattern does not match, but again is zero-width and doesn't advance the pointer. Negative lookahead is commonly used when we are parsing some complex pattern but want to rule out a special case. For example suppose we want to match, at the beginning of a line, any single word that doesn't start with "Volcano". We can use negative lookahead to do this:`

`/^(?!Volcano)[A-Za-z]+/` (2.15)

## 2.2 Words

**corpus**  
**corpora**

Before we talk about processing words, we need to decide what counts as a word. Let's start by looking at one particular **corpus** (plural **corpora**), a computer-readable collection of text or speech. For example the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.), assembled at Brown University in 1963–64 (Kučera and Francis, 1967). How many words are in the following Brown sentence?

He stepped out into the hall, was delighted to encounter  
a water brother.

This sentence has 13 words if we don't count punctuation marks as words, 15 if we count punctuation. Whether we treat period ("."), comma (","), and so on as words depends on the task. Punctuation is critical for finding boundaries of things (commas, periods, colons) and for identifying some aspects of meaning (question marks, exclamation marks, quotation marks). For some tasks, like part-of-speech tagging or parsing or speech synthesis, we sometimes treat punctuation marks as if they were separate words.

The Switchboard corpus of American English telephone conversations between strangers was collected in the early 1990s; it contains 2430 conversations averaging 6 minutes each, totaling 240 hours of speech and about 3 million words (Godfrey et al., 1992). Such corpora of spoken language introduce other complications with regard to defining words. Let's look at one utterance from Switchboard; an **utterance** is the spoken correlate of a sentence:

I do uh main- mainly business data processing

**disfluency**  
**fragment**  
**filled pause**

This utterance has two kinds of **disfluencies**. The broken-off word *main-* is called a **fragment**. Words like *uh* and *um* are called **fillers** or **filled pauses**. Should we consider these to be words? Again, it depends on the application. If we are building a speech transcription system, we might want to eventually strip out the disfluencies.

But we also sometimes keep disfluencies around. Disfluencies like *uh* or *um* are actually helpful in speech recognition in predicting the upcoming word, because they may signal that the speaker is restarting the clause or idea, and so for speech recognition they are treated as regular words. Because different people use different disfluencies they can also be a cue to speaker identification. In fact [Clark and Fox Tree \(2002\)](#) showed that *uh* and *um* have different meanings. What do you think they are?

Perhaps most important, in thinking about what is a word, we need to distinguish two ways of talking about words that will be useful throughout the book. **Word types** are the number of distinct words in a corpus; if the set of words in the vocabulary is  $V$ , the number of types is the vocabulary size  $|V|$ . **Word instances** are the total number  $N$  of running words.<sup>1</sup> If we ignore punctuation, the following Brown sentence has 14 types and 16 instances:

They picnicked by the pool, then lay back on the grass and  
looked at the stars.

We still have decisions to make! For example, should we consider a capitalized string (like *They*) and one that is uncapitalized (like *they*) to be the same word **type**? The answer is that it depends on the task! *They* and *they* might be lumped together as the same type in some tasks, like speech recognition, where we care more about the sequence of words and less about the formatting, while for other tasks, such as deciding whether a particular word is a name of a person or location (named-entity tagging), capitalization is a useful feature and is retained. Sometimes we keep around two versions of a particular NLP model, one with capitalization and one without capitalization.

| Corpus                              | Types = $ V $ | Instances = $N$ |
|-------------------------------------|---------------|-----------------|
| Shakespeare                         | 31 thousand   | 884 thousand    |
| Brown corpus                        | 38 thousand   | 1 million       |
| Switchboard telephone conversations | 20 thousand   | 2.4 million     |
| COCA                                | 2 million     | 440 million     |
| Google n-grams                      | 13 million    | 1 trillion      |

**Figure 2.11** Rough numbers of wordform types and instances for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

How many words are there in English? When we speak about the number of words in the language, we are generally referring to word types. Fig. 2.11 shows the rough numbers of types and instances computed from some English corpora. The larger the corpora we look at, the more word types we find, and in fact this relationship between the number of types  $|V|$  and number of instances  $N$  is called **Herdan's Law** ([Herdan, 1960](#)) or **Heaps' Law** ([Heaps, 1978](#)) after its discoverers (in linguistics and information retrieval respectively). It is shown in Eq. 2.16, where  $k$  and  $\beta$  are positive constants, and  $0 < \beta < 1$ .

$$|V| = kN^\beta \quad (2.16)$$

The value of  $\beta$  depends on the corpus size and the genre, but at least for the large corpora in Fig. 2.11,  $\beta$  ranges from .67 to .75. Roughly then we can say that the

<sup>1</sup> In earlier tradition, and occasionally still, you might see word instances referred to as word *tokens*, but we now try to reserve the word *token* instead to mean the output of subword tokenization algorithms.

vocabulary size for a text goes up significantly faster than the square root of its length in words.

lemma  
wordform

It's sometimes useful to make a further distinction. Consider inflected forms like *cats* versus *cat*. We say these two words are different **wordforms** but have the same **lemma**. A **lemma** is a set of lexical forms having the same stem, and usually the same major part-of-speech. The **wordform** is the full inflected or derived form of the word. The two wordforms *cat* and *cats* thus have the same lemma, which we can represent as *cat*.

For morphologically complex languages like Arabic, we often need to deal with lemmatization. For most tasks in English, however, wordforms are sufficient, and when we talk about words in this book we almost always mean wordforms (although we will discuss basic algorithms for lemmatization and the related task of stemming below in Section 2.6). One of the situations even in English where we talk about lemmas is when we measure the number of words in a dictionary. Dictionary **entries** or **boldface forms** are a very rough approximation to (an upper bound on) the number of lemmas (since some lemmas have multiple boldface forms). The 1989 edition of the Oxford English Dictionary had 615,000 entries.

Finally, we should note that in practice, for many NLP applications (for example for neural language modeling) we don't actually use words as our internal unit of representation at all! We instead **tokenize** the input strings into **tokens**, which can be words but can also be only parts of words. We'll return to this tokenization question when we introduce the **BPE** algorithm in Section 2.5.2.

## 2.3 Corpora

Words don't appear out of nowhere. Any particular piece of text that we study is produced by one or more specific speakers or writers, in a specific dialect of a specific language, at a specific time, in a specific place, for a specific function.

AAE

Perhaps the most important dimension of variation is the language. NLP algorithms are most useful when they apply across many languages. The world has 7097 languages at the time of this writing, according to the online Ethnologue catalog (Simons and Fennig, 2018). It is important to test algorithms on more than one language, and particularly on languages with different properties; by contrast there is an unfortunate current tendency for NLP algorithms to be developed or tested just on English (Bender, 2019). Even when algorithms are developed beyond English, they tend to be developed for the official languages of large industrialized nations (Chinese, Spanish, Japanese, German etc.), but we don't want to limit tools to just these few languages. Furthermore, most languages also have multiple varieties, often spoken in different regions or by different social groups. Thus, for example, if we're processing text that uses features of African American English (AAE) or African American Vernacular English (AAVE)—the variations of English used by millions of people in African American communities (King 2020)—we must use NLP tools that function with features of those varieties. Twitter posts might use features often used by speakers of African American English, such as constructions like *iont* (*I don't* in Mainstream American English (MAE)), or *talmbout* corresponding to MAE *talking about*, both examples that influence word segmentation (Blodgett et al. 2016, Jones 2015).

MAE

code switching

It's also quite common for speakers or writers to use multiple languages in a single communicative act, a phenomenon called **code switching**. Code switching

is enormously common across the world; here are examples showing Spanish and (transliterated) Hindi code switching with English (Solorio et al. 2014, Jurgens et al. 2017):

- (2.17) Por primera vez veo a @username actually being hateful! it was beautiful:)  
*[For the first time I get to see @username actually being hateful! it was beautiful:]*
- (2.18) dost tha or ra- hega ... dont worry ... but dherya rakhe  
*[“he was and will remain a friend ... don’t worry ... but have faith”]*

Another dimension of variation is the genre. The text that our algorithms must process might come from newswire, fiction or non-fiction books, scientific articles, Wikipedia, or religious texts. It might come from spoken genres like telephone conversations, business meetings, police body-worn cameras, medical interviews, or transcripts of television shows or movies. It might come from work situations like doctors’ notes, legal text, or parliamentary or congressional proceedings.

Text also reflects the demographic characteristics of the writer (or speaker): their age, gender, race, socioeconomic class can all influence the linguistic properties of the text we are processing.

And finally, time matters too. Language changes over time, and for some languages we have good corpora of texts from different historical periods.

Because language is so situated, when developing computational models for language processing from a corpus, it’s important to consider who produced the language, in what context, for what purpose. How can a user of a dataset know all these details? The best way is for the corpus creator to build a **datasheet** (Geburu et al., 2020) or **data statement** (Bender et al., 2021) for each corpus. A datasheet specifies properties of a dataset like:

**Motivation:** Why was the corpus collected, by whom, and who funded it?

**Situation:** When and in what situation was the text written/spoken? For example, was there a task? Was the language originally spoken conversation, edited text, social media communication, monologue vs. dialogue?

**Language variety:** What language (including dialect/region) was the corpus in?

**Speaker demographics:** What was, e.g., the age or gender of the text’s authors?

**Collection process:** How big is the data? If it is a subsample how was it sampled? Was the data collected with consent? How was the data pre-processed, and what metadata is available?

**Annotation process:** What are the annotations, what are the demographics of the annotators, how were they trained, how was the data annotated?

**Distribution:** Are there copyright or other intellectual property restrictions?

## 2.4 Simple Unix Tools for Word Tokenization

text  
normalization

Before almost any natural language processing of a text, the text has to be normalized, a task called **text normalization**. At least three tasks are commonly applied as part of any normalization process:

1. Tokenizing (segmenting) words
2. Normalizing word formats
3. Segmenting sentences

In the next sections we walk through each of these tasks, but we'll first start with an easy, if somewhat naive version of word tokenization and normalization (and frequency computation) that can be accomplished for English solely in a single Unix command-line, inspired by [Church \(1994\)](#). We'll make use of some Unix commands: `tr`, used to systematically change particular characters in the input; `sort`, which sorts input lines in alphabetical order; and `uniq`, which collapses and counts adjacent identical lines.

For example let's begin with the 'complete words' of Shakespeare in one file, `sh.txt`. We can use `tr` to tokenize the words by changing every sequence of non-alphabetic characters to a newline ('A-Za-z' means alphabetic and the `-c` option complements to non-alphabet, so together they mean to change every non-alphabetic character into a newline. The `-s` ('squeeze') option is used to replace the result of multiple consecutive changes into a single output, so a series of non-alphabetic characters in a row would all be 'squeezed' into a single newline):

```
tr -sc 'A-Za-z' '\n' < sh.txt
```

The output of this command will be:

```
THE
SONNETS
by
William
Shakespeare
From
fairest
creatures
We
...
```

Now that there is one word per line, we can sort the lines, and pass them to `uniq -c` which will collapse and count them:

```
tr -sc 'A-Za-z' '\n' < sh.txt | sort | uniq -c
```

with the following output:

```
1945 A
72 AARON
19 ABBESS
25 Aaron
6 Abate
1 Abates
5 Abbess
6 Abbey
3 Abbot
...
```

Alternatively, we can collapse all the upper case to lower case:

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c
```

whose output is

```
14725 a
97 aaron
1 abaissiez
10 abandon
```

```

2 abandoned
2 abase
1 abash
14 abate
3 abated
3 abatement
...

```

Now we can sort again to find the frequent words. The `-n` option to `sort` means to sort numerically rather than alphabetically, and the `-r` option means to sort in reverse order (highest-to-lowest):

```
tr -sc 'A-Za-z' '\n' < sh.txt | tr A-Z a-z | sort | uniq -c | sort -n -r
```

The results show that the most frequent words in Shakespeare, as in any other corpus, are the short **function words** like articles, pronouns, prepositions:

```

27378 the
26084 and
22538 i
19771 to
17481 of
14725 a
13826 you
12489 my
11318 that
11112 in
...

```

Unix tools of this sort can be very handy in building quick word count statistics for any corpus in English. While in some versions of Unix these command-line tools also correctly handle Unicode characters and so can be used for many languages, in general for handling most languages outside English we use more sophisticated tokenization algorithms.

## 2.5 Word and Subword Tokenization

**tokenization** The simple Unix tools above were fine for getting rough word statistics but more sophisticated algorithms are generally necessary for **tokenization**, the task of segmenting running text into words. There are roughly two classes of tokenization algorithms. In top-down tokenization, we define a standard and implement rules to implement that kind of tokenization.

**subword tokens** But more commonly instead of using words as the input to NLP algorithms we break up words into **subword tokens**, which can be words or parts of words or even individual letters. These are derived via bottom-up tokenization, in which we use simple statistics of letter sequences to come up with the vocabulary of subword tokens, and break up the input into those subwords.

### 2.5.1 Top-down (rule-based) tokenization

While the Unix command sequence just removed all the numbers and punctuation, for most NLP applications we'll need to keep these in our tokenization. We often

want to break off punctuation as a separate token; commas are a useful piece of information for parsers, and periods help indicate sentence boundaries. But we'll often want to keep the punctuation that occurs word internally, in examples like *m.p.h.*, *Ph.D.*, *AT&T*, and *cap'n*. Special characters and numbers will need to be kept in prices (\$45.55) and dates (01/02/06); we don't want to segment that price into separate tokens of "45" and "55". And there are URLs (<https://www.stanford.edu>), Twitter hashtags (#nlproc), or email addresses (someone@cs.colorado.edu).

Number expressions introduce complications; in addition to appearing at word boundaries, commas appear inside numbers in English, every three digits: 555,500.50. Tokenization differs by language; languages like Spanish, French, and German, for example, use a comma to mark the decimal point, and spaces (or sometimes periods) where English puts commas, for example, 555 500,50.

**clitic** A tokenizer can also be used to expand **clitic** contractions that are marked by apostrophes, converting *what're* to the two tokens *what are*, and *we're* to *we are*. A clitic is a part of a word that can't stand on its own, and can only occur when it is attached to another word. Such contractions occur in other alphabetic languages, including French pronouns (*j'ai* and articles *l'homme*).

Depending on the application, tokenization algorithms may also tokenize multiword expressions like *New York* or *rock 'n' roll* as a single token, which requires a multiword expression dictionary of some sort. Tokenization is thus intimately tied up with **named entity recognition**, the task of detecting names, dates, and organizations (Chapter 17).

**Penn Treebank tokenization** One commonly used tokenization standard is known as the **Penn Treebank tokenization** standard, used for the parsed corpora (treebanks) released by the Linguistic Data Consortium (LDC), the source of many useful datasets. This standard separates out clitics (*doesn't* becomes *does* plus *n't*), keeps hyphenated words together, and separates out all punctuation (to save space we're showing visible spaces '\_' between tokens, although newlines is a more common output):

**Input:** "The San Francisco-based restaurant," they said,  
"doesn't charge \$10".  
**Output:** "\_The\_San\_Francisco-based\_restaurant\_,\_"\_they\_said\_,\_  
"\_does\_n't\_charge\_\$10\_"\_.

In practice, since tokenization is run before any other language processing, it needs to be very fast. For word tokenization we generally use deterministic algorithms based on regular expressions compiled into efficient finite state automata. For example, Fig. 2.12 shows a basic regular expression that can be used to tokenize English with the `nltk.regexp_tokenize` function of the Python-based Natural Language Toolkit (NLTK) (Bird et al. 2009; <https://www.nltk.org>).

Carefully designed deterministic algorithms can deal with the ambiguities that arise, such as the fact that the apostrophe needs to be tokenized differently when used as a genitive marker (as in *the book's cover*), a quotative as in *'The other class'*, *she said*, or in clitics like *they're*.

**hanzi** Word tokenization is more complex in languages like written Chinese, Japanese, and Thai, which do not use spaces to mark potential word-boundaries. In Chinese, for example, words are composed of characters (called **hanzi** in Chinese). Each character generally represents a single unit of meaning (called a **morpheme**) and is pronounceable as a single syllable. Words are about 2.4 characters long on average. But deciding what counts as a word in Chinese is complex. For example, consider the following sentence:

```

>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     (?:[A-Z]\.)+        # abbreviations, e.g. U.S.A.
...     | \w+(?:-\w+)*      # words with optional internal hyphens
...     | \$?\d+(?:\.\d+)?%? # currency, percentages, e.g. $12.40, 82%
...     | \.\.\.           # ellipsis
...     | [][.,;"'()?()_-' ] # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']

```

**Figure 2.12** A Python trace of regular expression tokenization in the NLTK Python-based natural language processing toolkit (Bird et al., 2009), commented for readability; the (?x) verbose flag tells Python to strip comments and whitespace. Figure from Chapter 3 of Bird et al. (2009).

(2.19) 姚明进入总决赛            yáo míng jìn rù zǒng jué sài  
 “Yao Ming reaches the finals”

As Chen et al. (2017b) point out, this could be treated as 3 words (‘Chinese Tree-bank’ segmentation):

(2.20) 姚明    进入    总决赛  
 YaoMing reaches finals

or as 5 words (‘Peking University’ segmentation):

(2.21) 姚   明   进   入   总   决   赛  
 Yao Ming reaches overall finals

Finally, it is possible in Chinese simply to ignore words altogether and use characters as the basic elements, treating the sentence as a series of 7 characters:

(2.22) 姚   明   进   入   总   决   赛  
 Yao Ming enter enter overall decision game

In fact, for most Chinese NLP tasks it turns out to work better to take characters rather than words as input, since characters are at a reasonable semantic level for most applications, and since most word standards, by contrast, result in a huge vocabulary with large numbers of very rare words (Li et al., 2019b).

word  
segmentation

However, for Japanese and Thai the character is too small a unit, and so algorithms for **word segmentation** are required. These can also be useful for Chinese in the rare situations where word rather than character boundaries are required. For these situations we can use the subword tokenization algorithms introduced in the next section.

## 2.5.2 Byte-Pair Encoding: A Bottom-up Tokenization Algorithm

There is a third option to tokenizing text, one that is most commonly used by large language models. Instead of defining tokens as words (whether delimited by spaces or more complex algorithms), or as characters (as in Chinese), we can use our data to automatically tell us what the tokens should be. This is especially useful in dealing with unknown words, an important problem in language processing. As we will see in the next chapter, NLP algorithms often learn some facts about language from one corpus (a **training** corpus) and then use these facts to make decisions about a separate **test** corpus and its language. Thus if our training corpus contains, say the

words *low*, *new*, *newer*, but not *lower*, then if the word *lower* appears in our test corpus, our system will not know what to do with it.

subwords

To deal with this unknown word problem, modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called **subwords**. Subwords can be arbitrary substrings, or they can be meaning-bearing units like the morphemes *-est* or *-er*. (A morpheme is the smallest meaning-bearing unit of a language; for example the word *unwashable* has the morphemes *un-*, *wash*, and *-able*.) In modern tokenization schemes, most tokens are words, but some tokens are frequently occurring morphemes or other subwords like *-er*. Every unseen word like *lower* can thus be represented by some sequence of known subword units, such as *low* and *er*, or even as a sequence of individual letters if necessary.

Most tokenization schemes have two parts: a **token learner**, and a **token segmenter**. The token learner takes a raw training corpus (sometimes roughly pre-separated into words, for example by whitespace) and induces a vocabulary, a set of tokens. The token segmenter takes a raw test sentence and segments it into the tokens in the vocabulary. Two algorithms are widely used: **byte-pair encoding** (Sennrich et al., 2016), and **unigram language modeling** (Kudo, 2018). There is also a **SentencePiece** library that includes implementations of both of these (Kudo and Richardson, 2018a), and people often use the name **SentencePiece** to simply mean **unigram language modeling** tokenization.

BPE

In this section we introduce the simplest of the three, the **byte-pair encoding** or **BPE** algorithm (Sennrich et al., 2016); see Fig. 2.13. The BPE token learner begins with a vocabulary that is just the set of all individual characters. It then examines the training corpus, chooses the two symbols that are most frequently adjacent (say ‘A’, ‘B’), adds a new merged symbol ‘AB’ to the vocabulary, and replaces every adjacent ‘A’ ‘B’ in the corpus with the new ‘AB’. It continues to count and merge, creating new longer and longer character strings, until  $k$  merges have been done creating  $k$  novel tokens;  $k$  is thus a parameter of the algorithm. The resulting vocabulary consists of the original set of characters plus  $k$  new symbols.

The algorithm is usually run inside words (not merging across word boundaries), so the input corpus is first white-space-separated to give a set of strings, each corresponding to the characters of a word, plus a special end-of-word symbol  $\_$ , and its counts. Let’s see its operation on the following tiny input corpus of 18 word tokens with counts for each word (the word *low* appears 5 times, the word *newer* 6 times, and so on), which would have a starting vocabulary of 11 letters:

| corpus          | vocabulary                          |
|-----------------|-------------------------------------|
| 5 l o w _       | $\_$ , d, e, i, l, n, o, r, s, t, w |
| 2 l o w e s t _ |                                     |
| 6 n e w e r _   |                                     |
| 3 w i d e r _   |                                     |
| 2 n e w _       |                                     |

The BPE algorithm first counts all pairs of adjacent symbols: the most frequent is the pair *e r* because it occurs in *newer* (frequency of 6) and *wider* (frequency of 3) for a total of 9 occurrences.<sup>2</sup> We then merge these symbols, treating *er* as one symbol, and count again:

<sup>2</sup> Note that there can be ties; we could have instead chosen to merge *r \_* first, since that also has a frequency of 9.

| corpus          | vocabulary                          |
|-----------------|-------------------------------------|
| 5 l o w _       | _, d, e, i, l, n, o, r, s, t, w, er |
| 2 l o w e s t _ |                                     |
| 6 n e w er _    |                                     |
| 3 w i d er _    |                                     |
| 2 n e w _       |                                     |

Now the most frequent pair is er \_, which we merge; our system has learned that there should be a token for word-final er, represented as er\_:

| corpus          | vocabulary                               |
|-----------------|------------------------------------------|
| 5 l o w _       | _, d, e, i, l, n, o, r, s, t, w, er, er_ |
| 2 l o w e s t _ |                                          |
| 6 n e w er_     |                                          |
| 3 w i d er_     |                                          |
| 2 n e w _       |                                          |

Next n e (total count of 8) get merged to ne:

| corpus          | vocabulary                                   |
|-----------------|----------------------------------------------|
| 5 l o w _       | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne |
| 2 l o w e s t _ |                                              |
| 6 ne w er_      |                                              |
| 3 w i d er_     |                                              |
| 2 ne w _        |                                              |

If we continue, the next merges are:

| merge      | current vocabulary                                                       |
|------------|--------------------------------------------------------------------------|
| (ne, w)    | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new                        |
| (l, o)     | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo                    |
| (lo, w)    | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low               |
| (new, er_) | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_       |
| (low, _)   | _, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_ |

```

function BYTE-PAIR ENCODING(strings  $C$ , number of merges  $k$ ) returns vocab  $V$ 

 $V \leftarrow$  all unique characters in  $C$            # initial set of tokens is characters
for  $i = 1$  to  $k$  do                           # merge tokens  $k$  times
     $t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$ 
     $t_{NEW} \leftarrow t_L + t_R$                  # make new token by concatenating
     $V \leftarrow V + t_{NEW}$                        # update the vocabulary
    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$    # and update the corpus
return  $V$ 

```

**Figure 2.13** The token learner part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from [Bostrom and Durrett \(2020\)](#).

Once we've learned our vocabulary, the token segmenter is used to tokenize a test sentence. The token segmenter just runs on the merges we have learned from the training data on the test data. It runs them greedily, in the order we learned them. (Thus the frequencies in the test data don't play a role, just the frequencies in the training data). So first we segment each test sentence word into characters. Then we apply the first rule: replace every instance of e r in the test corpus with er, and

then the second rule: replace every instance of `er _` in the test corpus with `er_`, and so on. By the end, if the test corpus contained the character sequence `n e w e r _`, it would be tokenized as a full word. But the characters of a new (unknown) word like `l o w e r _` would be merged into the two tokens `low er_`.

Of course in real settings BPE is run with many thousands of merges on a very large input corpus. The result is that most words will be represented as full symbols, and only the very rare words (and unknown words) will have to be represented by their parts.

## 2.6 Word Normalization, Lemmatization and Stemming

normalization  
case folding

Word **normalization** is the task of putting words or tokens in a standard format. The simplest case of word normalization is **case folding**. Mapping everything to lower case means that *Woodchuck* and *woodchuck* are represented identically, which is very helpful for generalization in many tasks, such as information retrieval or speech recognition. For sentiment analysis and other text classification tasks, information extraction, and machine translation, by contrast, case can be quite helpful and case folding is generally not done. This is because maintaining the difference between, for example, *US* the country and *us* the pronoun can outweigh the advantage in generalization that case folding would have provided for other words. Sometimes we produce both cased (i.e. including both upper and lower case words or tokens) and uncased versions of language models.

Systems that use BPE or other kinds of bottom-up tokenization may do no further word normalization. In other NLP systems, we may want to do further normalizations, like choosing a single normal form for words with multiple forms like *USA* and *US* or *uh-huh* and *uhhuh*. This standardization may be valuable, despite the spelling information that is lost in the normalization process. For information retrieval or information extraction about the *US*, we might want to see information from documents whether they mention the *US* or the *USA*.

### 2.6.1 Lemmatization

lemmatization

For other natural language processing situations we also want two morphologically different forms of a word to behave similarly. For example in web search, someone may type the string *woodchucks* but a useful system might want to also return pages that mention *woodchuck* with no *s*. This is especially common in morphologically complex languages like Polish, where for example the word *Warsaw* has different endings when it is the subject (*Warszawa*), or after a preposition like “in Warsaw” (*w Warszawie*), or “to Warsaw” (*do Warszawy*), and so on. **Lemmatization** is the task of determining that two words have the same root, despite their surface differences. The words *am*, *are*, and *is* have the shared lemma *be*; the words *dinner* and *dinners* both have the lemma *dinner*. Lemmatizing each of these forms to the same lemma will let us find all mentions of words in Polish like *Warsaw*. The lemmatized form of a sentence like *He is reading detective stories* would thus be *He be read detective story*.

morpheme  
stem

How is lemmatization done? The most sophisticated methods for lemmatization involve complete **morphological parsing** of the word. **Morphology** is the study of the way words are built up from smaller meaning-bearing units called **morphemes**. Two broad classes of morphemes can be distinguished: **stems**—the central mor-

**affix** pheme of the word, supplying the main meaning—and **affixes**—adding “additional” meanings of various kinds. So, for example, the word *fox* consists of one morpheme (the morpheme *fox*) and the word *cats* consists of two: the morpheme *cat* and the morpheme *-s*. A morphological parser takes a word like *cats* and parses it into the two morphemes *cat* and *s*, or parses a Spanish word like *amaren* (‘if in the future they would love’) into the morpheme *amar* ‘to love’, and the morphological features *3PL* (third person plural) and *future subjunctive*.

### Stemming: The Porter Stemmer

**stemming**  
**Porter stemmer**

Lemmatization algorithms can be complex. For this reason we sometimes make use of a simpler but cruder method, which mainly consists of chopping off word-final affixes. This naive version of morphological analysis is called **stemming**. For example, the classic **Porter stemmer** (Porter, 1980), when applied to the following paragraph:

This was not the map we found in Billy Bones’s chest, but an accurate copy, complete in all things—names and heights and soundings—with the single exception of the red crosses and the written notes.

produces the following stemmed output:

Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note

The algorithm is based on rewrite rules run in series, with the output of each pass fed as input to the next pass. Some sample rules (more at <https://tartarus.org/martin/PorterStemmer/>):

ATIONAL → ATE (e.g., relational → relate)  
 ING →  $\epsilon$  if the stem contains a vowel (e.g., motoring → motor)  
 SSES → SS (e.g., grasses → grass)

Simple stemmers can be useful in cases where we need to collapse across different variants of the same lemma. Nonetheless, they are less commonly used in modern systems since they commit errors of both over-generalizing (lemmatizing *policy* to *police*) and under-generalizing (not lemmatizing *European* to *Europe*) (Krovetz, 1993).

## 2.7 Sentence Segmentation

**sentence segmentation**

**Sentence segmentation** is another important step in text processing. The most useful cues for segmenting a text into sentences are punctuation, like periods, question marks, and exclamation points. Question marks and exclamation points are relatively unambiguous markers of sentence boundaries. Periods, on the other hand, are more ambiguous. The period character “.” is ambiguous between a sentence boundary marker and a marker of abbreviations like *Mr.* or *Inc.* The previous sentence that you just read showed an even more complex case of this ambiguity, in which the final period of *Inc.* marked both an abbreviation and the sentence boundary marker. For this reason, sentence tokenization and word tokenization may be addressed jointly.

In general, sentence tokenization methods work by first deciding (based on rules or machine learning) whether a period is part of the word or is a sentence-boundary marker. An abbreviation dictionary can help determine whether the period is part of a commonly used abbreviation; the dictionaries can be hand-built or machine-learned (Kiss and Strunk, 2006), as can the final sentence splitter. In the Stanford CoreNLP toolkit (Manning et al., 2014), for example sentence splitting is rule-based, a deterministic consequence of tokenization; a sentence ends when a sentence-ending punctuation (., !, or ?) is not already grouped with other characters into a token (such as for an abbreviation or number), optionally followed by additional final quotes or brackets.

## 2.8 Minimum Edit Distance

Much of natural language processing is concerned with measuring how similar two strings are. For example in spelling correction, the user typed some erroneous string—let’s say *graffe*—and we want to know what the user meant. The user probably intended a word that is similar to *graffe*. Among candidate similar words, the word *giraffe*, which differs by only one letter from *graffe*, seems intuitively to be more similar than, say *grail* or *graf*, which differ in more letters. Another example comes from **coreference**, the task of deciding whether two strings such as the following refer to the same entity:

Stanford Arizona Cactus Garden  
Stanford University Arizona Cactus Garden

Again, the fact that these two strings are very similar (differing by only one word) seems like useful evidence for deciding that they might be coreferent. Finally, string similarity is commonly used to measure the quality of the transcription produced by a speech recognition system, by asking how similar (in words) the transcript is to a reference transcript. A system whose transcript is off by many words is measurably worse than one which is only off by a few words.

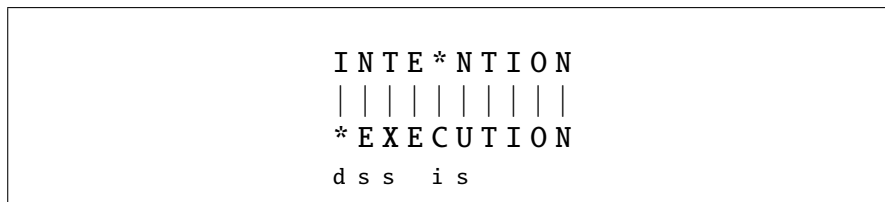
minimum edit  
distance

**Edit distance** gives us a way to quantify these intuitions about string similarity. More formally, the **minimum edit distance** between two strings is defined as the minimum number of editing operations (operations like insertion, deletion, substitution) needed to transform one string into another.

alignment

The gap between *intention* and *execution*, for example, is 5 (delete an *i*, substitute *e* for *n*, substitute *x* for *t*, insert *c*, substitute *u* for *n*). It’s much easier to see this by looking at the most important visualization for string distances, an **alignment** between the two strings, shown in Fig. 2.14. Given two sequences, an **alignment** is a correspondence between substrings of the two sequences. Thus, we say *I aligns* with the empty string, *N* with *E*, and so on. Beneath the aligned strings is another representation; a series of symbols expressing an **operation list** for converting the top string into the bottom string: **d** for deletion, **s** for substitution, **i** for insertion.

We can also assign a particular cost or weight to each of these operations. The **Levenshtein** distance between two sequences is the simplest weighting factor in which each of the three operations has a cost of 1 (Levenshtein, 1966)—we assume that the substitution of a letter for itself, for example, *t* for *t*, has zero cost. The Levenshtein distance between *intention* and *execution* is 5. Levenshtein also proposed an alternative version of his metric in which each insertion or deletion has a cost of 1 and substitutions are not allowed. (This is equivalent to allowing substitution, but

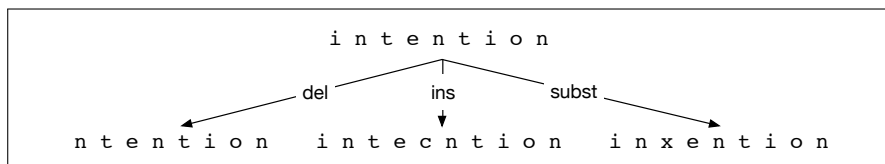


**Figure 2.14** Representing the minimum edit distance between two strings as an **alignment**. The final row gives the operation list for converting the top string into the bottom string: d for deletion, s for substitution, i for insertion.

giving each substitution a cost of 2 since any substitution can be represented by one insertion and one deletion). Using this version, the Levenshtein distance between *intention* and *execution* is 8.

### 2.8.1 The Minimum Edit Distance Algorithm

How do we find the minimum edit distance? We can think of this as a search task, in which we are searching for the shortest path—a sequence of edits—from one string to another.



**Figure 2.15** Finding the edit distance viewed as a search problem

The space of all possible edits is enormous, so we can't search naively. However, lots of distinct edit paths will end up in the same state (string), so rather than recomputing all those paths, we could just remember the shortest path to a state each time we saw it. We can do this by using **dynamic programming**. Dynamic programming is the name for a class of algorithms, first introduced by [Bellman \(1957\)](#), that apply a table-driven method to solve problems by combining solutions to subproblems. Some of the most commonly used algorithms in natural language processing make use of dynamic programming, such as the **Viterbi** algorithm (Chapter 17) and the **CKY** algorithm for parsing (Chapter 18).

The intuition of a dynamic programming problem is that a large problem can be solved by properly combining the solutions to various subproblems. Consider the shortest path of transformed words that represents the minimum edit distance between the strings *intention* and *execution* shown in Fig. 2.16.

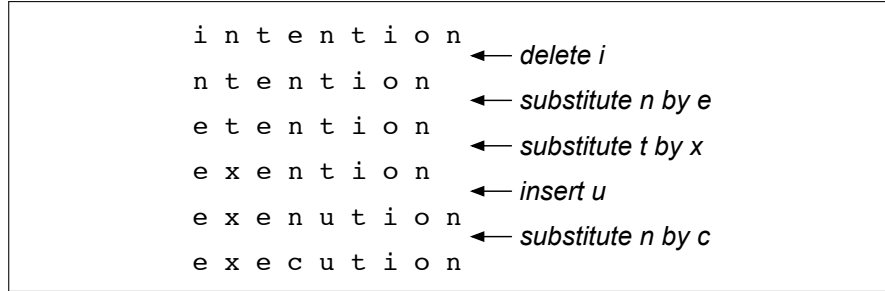
Imagine some string (perhaps it is *exention*) that is in this optimal path (whatever it is). The intuition of dynamic programming is that if *exention* is in the optimal operation list, then the optimal sequence must also include the optimal path from *intention* to *exention*. Why? If there were a shorter path from *intention* to *exention*, then we could use it instead, resulting in a shorter overall path, and the optimal sequence wouldn't be optimal, thus leading to a contradiction.

The **minimum edit distance algorithm** was named by [Wagner and Fischer \(1974\)](#) but independently discovered by many people (see the Historical Notes section of Chapter 17).

Let's first define the minimum edit distance between two strings. Given two strings, the source string  $X$  of length  $n$ , and target string  $Y$  of length  $m$ , we'll define

dynamic  
programming

minimum edit  
distance  
algorithm



**Figure 2.16** Path from *intention* to *execution*.

$D[i, j]$  as the edit distance between  $X[1..i]$  and  $Y[1..j]$ , i.e., the first  $i$  characters of  $X$  and the first  $j$  characters of  $Y$ . The edit distance between  $X$  and  $Y$  is thus  $D[n, m]$ .

We'll use dynamic programming to compute  $D[n, m]$  bottom up, combining solutions to subproblems. In the base case, with a source substring of length  $i$  but an empty target string, going from  $i$  characters to 0 requires  $i$  deletes. With a target substring of length  $j$  but an empty source going from 0 characters to  $j$  characters requires  $j$  inserts. Having computed  $D[i, j]$  for small  $i, j$  we then compute larger  $D[i, j]$  based on previously computed smaller values. The value of  $D[i, j]$  is computed by taking the minimum of the three possible paths through the matrix which arrive there:

$$D[i, j] = \min \begin{cases} D[i-1, j] + \text{del-cost}(\text{source}[i]) \\ D[i, j-1] + \text{ins-cost}(\text{target}[j]) \\ D[i-1, j-1] + \text{sub-cost}(\text{source}[i], \text{target}[j]) \end{cases} \quad (2.23)$$

We mentioned above two versions of Levenshtein distance, one in which substitutions cost 1 and one in which substitutions cost 2 (i.e., are equivalent to an insertion plus a deletion). Let's here use that second version of Levenshtein distance in which the insertions and deletions each have a cost of 1 ( $\text{ins-cost}(\cdot) = \text{del-cost}(\cdot) = 1$ ), and substitutions have a cost of 2 (except substitution of identical letters has zero cost). Under this version of Levenshtein, the computation for  $D[i, j]$  becomes:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 \\ D[i, j-1] + 1 \\ D[i-1, j-1] + \begin{cases} 2; & \text{if } \text{source}[i] \neq \text{target}[j] \\ 0; & \text{if } \text{source}[i] = \text{target}[j] \end{cases} \end{cases} \quad (2.24)$$

The algorithm is summarized in Fig. 2.17; Fig. 2.18 shows the results of applying the algorithm to the distance between *intention* and *execution* with the version of Levenshtein in Eq. 2.24.

**Alignment** Knowing the minimum edit distance is useful for algorithms like finding potential spelling error corrections. But the edit distance algorithm is important in another way; with a small change, it can also provide the minimum cost **alignment** between two strings. Aligning two strings is useful throughout speech and language processing. In speech recognition, minimum edit distance alignment is used to compute the word error rate (Chapter 16). Alignment plays a role in machine translation, in which sentences in a parallel corpus (a corpus with a text in two languages) need to be matched to each other.

To extend the edit distance algorithm to produce an alignment, we can start by visualizing an alignment as a path through the edit distance matrix. Figure 2.19

```

function MIN-EDIT-DISTANCE(source, target) returns min-distance

     $n \leftarrow \text{LENGTH}(\textit{source})$ 
     $m \leftarrow \text{LENGTH}(\textit{target})$ 
    Create a distance matrix  $D[n+1, m+1]$ 

    # Initialization: the zeroth row and column is the distance from the empty string
     $D[0,0] = 0$ 
    for each row  $i$  from 1 to  $n$  do
         $D[i,0] \leftarrow D[i-1,0] + \textit{del-cost}(\textit{source}[i])$ 
    for each column  $j$  from 1 to  $m$  do
         $D[0,j] \leftarrow D[0,j-1] + \textit{ins-cost}(\textit{target}[j])$ 

    # Recurrence relation:
    for each row  $i$  from 1 to  $n$  do
        for each column  $j$  from 1 to  $m$  do
             $D[i,j] \leftarrow \text{MIN}( D[i-1,j] + \textit{del-cost}(\textit{source}[i]),$ 
                                $D[i-1,j-1] + \textit{sub-cost}(\textit{source}[i], \textit{target}[j]),$ 
                                $D[i,j-1] + \textit{ins-cost}(\textit{target}[j]))$ 

    # Termination
    return  $D[n,m]$ 

```

**Figure 2.17** The minimum edit distance algorithm, an example of the class of dynamic programming algorithms. The various costs can either be fixed (e.g.,  $\forall x, \textit{ins-cost}(x) = 1$ ) or can be specific to the letter (to model the fact that some letters are more likely to be inserted than others). We assume that there is no cost for substituting a letter for itself (i.e.,  $\textit{sub-cost}(x, x) = 0$ ).

| Src\Tar | # | e | x | e  | c  | u  | t  | i  | o  | n  |
|---------|---|---|---|----|----|----|----|----|----|----|
| #       | 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| i       | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 6  | 7  | 8  |
| n       | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 7  | 8  | 7  |
| t       | 3 | 4 | 5 | 6  | 7  | 8  | 7  | 8  | 9  | 8  |
| e       | 4 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 | 9  |
| n       | 5 | 4 | 5 | 6  | 7  | 8  | 9  | 10 | 11 | 10 |
| t       | 6 | 5 | 6 | 7  | 8  | 9  | 8  | 9  | 10 | 11 |
| i       | 7 | 6 | 7 | 8  | 9  | 10 | 9  | 8  | 9  | 10 |
| o       | 8 | 7 | 8 | 9  | 10 | 11 | 10 | 9  | 8  | 9  |
| n       | 9 | 8 | 9 | 10 | 11 | 12 | 11 | 10 | 9  | 8  |

**Figure 2.18** Computation of minimum edit distance between *intention* and *execution* with the algorithm of Fig. 2.17, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

shows this path with boldfaced cells. Each boldfaced cell represents an alignment of a pair of letters in the two strings. If two boldfaced cells occur in the same row, there will be an insertion in going from the source to the target; two boldfaced cells in the same column indicate a deletion.

Figure 2.19 also shows the intuition of how to compute this alignment path. The computation proceeds in two steps. In the first step, we augment the minimum edit distance algorithm to store backpointers in each cell. The backpointer from a cell points to the previous cell (or cells) that we came from in entering the current cell. We've shown a schematic of these backpointers in Fig. 2.19. Some cells have mul-

## backtrace

multiple backpointers because the minimum extension could have come from multiple previous cells. In the second step, we perform a **backtrace**. In a backtrace, we start from the last cell (at the final row and column), and follow the pointers back through the dynamic programming matrix. Each complete path between the final cell and the initial cell is a minimum distance alignment. Exercise 2.7 asks you to modify the minimum edit distance algorithm to store the pointers and compute the backtrace to output an alignment.

|   | #   | e    | x    | e     | c     | u     | t    | i     | o     | n    |
|---|-----|------|------|-------|-------|-------|------|-------|-------|------|
| # | 0   | ← 1  | ← 2  | ← 3   | ← 4   | ← 5   | ← 6  | ← 7   | ← 8   | ← 9  |
| i | ↑ 1 | ↖↗ 2 | ↖↗ 3 | ↖↗ 4  | ↖↗ 5  | ↖↗ 6  | ↖↗ 7 | ↖ 6   | ← 7   | ← 8  |
| n | ↑ 2 | ↖↗ 3 | ↖↗ 4 | ↖↗ 5  | ↖↗ 6  | ↖↗ 7  | ↖↗ 8 | ↑ 7   | ↖↗ 8  | ↖ 7  |
| t | ↑ 3 | ↖↗ 4 | ↖↗ 5 | ↖↗ 6  | ↖↗ 7  | ↖↗ 8  | ↖ 7  | ↖↗ 8  | ↖↗ 9  | ↑ 8  |
| e | ↑ 4 | ↖ 3  | ← 4  | ↖↗ 5  | ← 6   | ← 7   | ↖↗ 8 | ↖↗ 9  | ↖↗ 10 | ↑ 9  |
| n | ↑ 5 | ↑ 4  | ↖↗ 5 | ↖↗ 6  | ↖↗ 7  | ↖↗ 8  | ↖↗ 9 | ↖↗ 10 | ↖↗ 11 | ↖ 10 |
| t | ↑ 6 | ↑ 5  | ↖↗ 6 | ↖↗ 7  | ↖↗ 8  | ↖↗ 9  | ↖ 8  | ← 9   | ← 10  | ↖ 11 |
| i | ↑ 7 | ↑ 6  | ↖↗ 7 | ↖↗ 8  | ↖↗ 9  | ↖↗ 10 | ↑ 9  | ↖ 8   | ← 9   | ← 10 |
| o | ↑ 8 | ↑ 7  | ↖↗ 8 | ↖↗ 9  | ↖↗ 10 | ↖↗ 11 | ↑ 10 | ↑ 9   | ↖ 8   | ← 9  |
| n | ↑ 9 | ↑ 8  | ↖↗ 9 | ↖↗ 10 | ↖↗ 11 | ↖↗ 12 | ↑ 11 | ↑ 10  | ↑ 9   | ↖ 8  |

**Figure 2.19** When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings, again using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions. Diagram design after [Gusfield \(1997\)](#).

While we worked our example with simple Levenshtein distance, the algorithm in Fig. 2.17 allows arbitrary weights on the operations. For spelling correction, for example, substitutions are more likely to happen between letters that are next to each other on the keyboard. The **Viterbi** algorithm is a probabilistic extension of minimum edit distance. Instead of computing the “minimum edit distance” between two strings, Viterbi computes the “maximum probability alignment” of one string with another. We’ll discuss this more in Chapter 17.

## 2.9 Summary

This chapter introduced a fundamental tool in language processing, the **regular expression**, and showed how to perform basic **text normalization** tasks including **word segmentation** and **normalization**, **sentence segmentation**, and **stemming**. We also introduced the important **minimum edit distance** algorithm for comparing strings. Here’s a summary of the main points we covered about these ideas:

- The **regular expression** language is a powerful tool for pattern-matching.
- Basic operations in regular expressions include **concatenation** of symbols, **disjunction** of symbols (`[]`, `|`), **counters** (`*`, `+`, and `{n,m}`), **anchors** (`^`, `$`) and precedence operators (`(,)`).
- **Word tokenization and normalization** are generally done by cascades of simple regular expression substitutions or finite automata.
- The **Porter algorithm** is a simple and efficient way to do **stemming**, stripping off affixes. It does not have high accuracy but may be useful for some tasks.

- The **minimum edit distance** between two strings is the minimum number of operations it takes to edit one into the other. Minimum edit distance can be computed by **dynamic programming**, which also results in an **alignment** of the two strings.

## Bibliographical and Historical Notes

Kleene 1951; 1956 first defined regular expressions and the finite automaton, based on the McCulloch-Pitts neuron. Ken Thompson was one of the first to build regular expressions compilers into editors for text searching (Thompson, 1968). His editor *ed* included a command “g/regular expression/p”, or Global Regular Expression Print, which later became the Unix *grep* utility.

Text normalization algorithms have been applied since the beginning of the field. One of the earliest widely used stemmers was Lovins (1968). Stemming was also applied early to the digital humanities, by Packard (1973), who built an affix-stripping morphological parser for Ancient Greek. Currently a wide variety of code for tokenization and normalization is available, such as the Stanford Tokenizer (<https://nlp.stanford.edu/software/tokenizer.shtml>) or specialized tokenizers for Twitter (O'Connor et al., 2010), or for sentiment (<http://sentiment.christopherpotts.net/tokenizing.html>). See Palmer (2012) for a survey of text preprocessing. NLTK is an essential tool that offers both useful Python libraries (<https://www.nltk.org>) and textbook descriptions (Bird et al., 2009) of many algorithms including text normalization and corpus interfaces.

For more on Herdan’s law and Heaps’ Law, see Herdan (1960, p. 28), Heaps (1978), Egghe (2007) and Baayen (2001); For more on edit distance, see Gusfield (1997). Our example measuring the edit distance from ‘intention’ to ‘execution’ was adapted from Kruskal (1983). There are various publicly available packages to compute edit distance, including Unix *diff* and the NIST *sc1ite* program (NIST, 2005).

In his autobiography Bellman (1984) explains how he originally came up with the term *dynamic programming*:

“...The 1950s were not good years for mathematical research. [the] Secretary of Defense ...had a pathological fear and hatred of the word, research... I decided therefore to use the word, “programming”. I wanted to get across the idea that this was dynamic, this was multi-stage... I thought, let’s ... take a word that has an absolutely precise meaning, namely dynamic... it’s impossible to use the word, dynamic, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It’s impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to.”

## Exercises

- 2.1 Write regular expressions for the following languages.
1. the set of all alphabetic strings;
  2. the set of all lower case alphabetic strings ending in a *b*;

3. the set of all strings from the alphabet  $a, b$  such that each  $a$  is immediately preceded by and immediately followed by a  $b$ ;
- 2.2** Write regular expressions for the following languages. By “word”, we mean an alphabetic string separated from other words by whitespace, any relevant punctuation, line breaks, and so forth.
1. the set of all strings with two consecutive repeated words (e.g., “Humbert Humbert” and “the the” but not “the bug” or “the big bug”);
  2. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word;
  3. all strings that have both the word *grotto* and the word *raven* in them (but not, e.g., words like *grottos* that merely *contain* the word *grotto*);
  4. write a pattern that places the first word of an English sentence in a register. Deal with punctuation.
- 2.3** Implement an ELIZA-like program, using substitutions such as those described on page 12. You might want to choose a different domain than a Rogerian psychologist, although keep in mind that you would need a domain in which your program can legitimately engage in a lot of simple repetition.
- 2.4** Compute the edit distance (using insertion cost 1, deletion cost 1, substitution cost 1) of “leda” to “deal”. Show your work (using the edit distance grid).
- 2.5** Figure out whether *drive* is closer to *brief* or to *divers* and what the edit distance is to each. You may use any version of *distance* that you like.
- 2.6** Now implement a minimum edit distance algorithm and use your hand-computed results to check your code.
- 2.7** Augment the minimum edit distance algorithm to output an alignment; you will need to store pointers and add a stage to compute the backtrace.